

 MLOPS

LLM Observability and Tracing

If you cannot see inside your LLM pipeline, you cannot debug it, optimize it, or trust it. Observability is not optional in production AI.

4 pillars every production LLM pipeline must have

 TRACING

 COST ATTRIBUTION

 LATENCY

 PROMPT VERSIONING

OpenTelemetry + RAGAS + LangSmith + Prometheus. Real production code.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

The 4 Pillars of LLM Observability

Distributed Tracing

Every query generates a trace spanning retrieval, reranking, and generation. Each step gets a timed span with inputs and outputs. Full audit trail per request.

Cost Attribution

Token counts and USD costs attributed to each feature, user, team, and experiment. Know exactly who is spending what, daily and monthly.

Latency Breakdown

p50/p95/p99 per pipeline component: retrieval, reranking, generation, total. Find the actual bottleneck, not just total response time.

Prompt Versioning

Every prompt template gets a SHA-256 hash. When quality changes, identify the exact prompt version responsible. Compare versions on identical queries.

 Without these 4 pillars, "something is wrong" is all you can say. With them, you know: "retrieval latency spiked 3x in the summarizer feature since 14:22 when prompt v2a3f deployed."



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Why LLM Pipelines **Are Hard to Debug**

Traditional software fails with stack traces and error codes. LLM pipelines fail silently: slow retrieval, bad context selection, subtle hallucinations, and cost overruns look identical from the outside.

☠ **Without Observability**

You see: "users say answers are getting worse." You have no idea if the problem is retrieval quality, context length, prompt version, model behavior, or embedding drift. Investigation takes days.

✓ **With Observability**

You see: "faithfulness score dropped from 0.88 to 0.71 on Wednesday at 15:00. Prompt version changed from abc123 to def456 at 14:55. Retrieval scores unchanged. Root cause: new prompt." Investigation takes minutes.

\$ **Cost Visibility**

Without attribution: "our OpenAI bill went from \$2k to \$8k this month." With attribution: "the new document summarizer feature accounts for 73% of the increase."



OpenTelemetry Span-Based Tracing

```
from opentelemetry import trace

tracer = trace.get_tracer("rag-pipeline")

async def full_rag_pipeline(query: str, user_id: str):
    with tracer.start_as_current_span("rag.full") as root:
        root.set_attributes({
            "user_id": user_id, "query_len": len(query)
        })
        with tracer.start_as_current_span("rag.retrieve") as s:
            chunks = await retrieve(query)
            s.set_attribute("chunks_returned", len(chunks))

            with tracer.start_as_current_span("rag.generate") as s:
                resp = await generate(query, chunks)
                s.set_attributes({
                    "tokens_in": resp.usage.prompt_tokens,
                    "tokens_out": resp.usage.completion_tokens
                })
            return resp.choices[0].message.content
```

Backends

Jaeger (self-hosted free). Tempo + Grafana (cloud-native). Datadog APM (managed). LangSmith (LLM-native). All consume OTLP traces.

Overhead

OpenTelemetry adds under 1ms overhead per span. Exporting is async. No measurable latency impact on user-facing requests.



LLMCallRecord and Cost Attribution

```

from dataclasses import dataclass
import threading

PRICING = { # USD per 1M tokens (input, output)
    "gpt-4o": (2.50, 10.00),
    "gpt-4o-mini": (0.15, 0.60),
    "claude-sonnet-4-6": (3.00, 15.00),
    "claude-haiku-4-5": (0.80, 4.00),
}

@dataclass
class LLMCallRecord:
    model: str; feature: str; user_id: str
    in_tokens: int; out_tokens: int

    def compute_cost(self) → float:
        p_in, p_out = PRICING[self.model]
        return (self.in_tokens * p_in + self.out_tokens * p_out) / 1_000_000

class CostTracker:
    def __init__(self): self._lock = threading.Lock(); self.spend = {}
    def record(self, rec: LLMCallRecord):
        cost = rec.compute_cost()
        with self._lock:
            self.spend[rec.feature] = self.spend.get(rec.feature, 0) + cost

```



Tag every LLM call with feature name. Run a daily cost report per feature. Stop being surprised at the end of the month.



AI Coach John

Follow



Repost

Per-Component Latency Breakdown

```
from contextlib import contextmanager
import time
from prometheus_client import Histogram

latency_hist = Histogram("llm_component_latency_ms", "Latency per component"
                        ["component"], buckets=[10,50,100,200,500,1000,3000])

@contextmanager
def timed(label: str, timings: dict):
    start = time.perf_counter()
    yield
    ms = (time.perf_counter() - start) * 1000
    timings[label] = ms
    latency_hist.labels(component=label).observe(ms)

# Usage in pipeline
timings = {}
with timed("retrieval", timings): chunks = await retrieve(query)
with timed("rerank", timings): chunks = await rerank(query, chunks)
with timed("generate", timings): answer = await generate(query, chunks)
```

🎯 Latency Targets

Retrieval p99: below 200ms. Rerank p99: below 100ms. Generation p99: below 3000ms. Total p99: below 4000ms.

🔔 Alert Rule

Alert when any component exceeds 2x its 7-day p99 baseline. Separate alerts per component (not just total latency).



Prompt Hash Tracking with SHA-256

```
import hashlib
from dataclasses import dataclass

@dataclass
class PromptTemplate:
    name: str
    template: str

    def version_hash(self) → str:
        return hashlib.sha256(self.template.encode()).hexdigest()[:8]

    def render(self, **kwargs) → str:
        return self.template.format(**kwargs)

rag_prompt = PromptTemplate(
    name="rag_answer",
    template="Context:\n{context}\n\nQuestion: {query}\nAnswer:"
)

version_tag = f"{rag_prompt.name}@{rag_prompt.version_hash()}"
# → "rag_answer@3fa8c21b" (logged in every trace)

# In your trace / span attributes:
span.set_attribute("prompt_version", version_tag)
```

Q Why This Matters for Debugging

Quality drops at 14:55 on Wednesday. Query your trace store for all traces where `prompt_version = "rag_answer@3fa8c21b"`. Compare quality metrics (RAGAS faithfulness) before and after that version deployed. Root cause in 10 minutes.



AI Coach John

Follow



 Repost

RAGAS Async Quality Evaluation

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_recall, context_precision
)
from datasets import Dataset
import asyncio

async def evaluate_and_log(prod_sample: list[dict]) → dict:
    dataset = Dataset.from_list([
        {"question": s["query"],
         "contexts": s["contexts"],
         "answer": s["answer"],
         "ground_truth": s["ground_truth"]}
        for s in prod_sample])

    scores = evaluate(dataset,
        metrics=[faithfulness, answer_relevancy,
                 context_recall, context_precision])
    await write_to_metrics_store(scores)
    return scores

# faithfulness below 0.80 → alert team immediately
```

 Run weekly on 100 randomly sampled production queries. Track all 4 metrics as time series. Alert when any drops more than 10% week over week.



AI Coach John

Follow



 Repost

User Feedback **Linked to Traces**

Link every user thumbs-up/thumbs-down feedback to the specific trace that generated the answer. This lets you analyze which pipeline conditions correlate with user satisfaction.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class FeedbackRequest(BaseModel):
    trace_id: str
    rating: int          # 1-5 scale
    comment: str = ""

@app.post("/feedback")
async def submit_feedback(req: FeedbackRequest):
    await db.upsert("feedback", {
        "trace_id": req.trace_id,
        "rating":   req.rating,
        "comment":  req.comment,
        "ts":       datetime.utcnow().isoformat()
    })
    # Join feedback with trace to get retrieval scores,
```

Correlation Analysis

Join feedback table with trace table on trace_id. Compute: avg rating by retrieval score bucket, avg rating by prompt version, avg rating by model. Find what drives satisfaction.



LangSmith Zero-Config LLM Tracing

LangSmith provides LLM-native tracing with zero boilerplate. Set three environment variables and every LLM call is automatically traced with inputs, outputs, tokens, and cost.

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "ls- ..."
os.environ["LANGCHAIN_PROJECT"] = "prod-rag-v2"

from langsmith import traceable
from openai import AsyncOpenAI

client = AsyncOpenAI()

@traceable(name="rag_generate", run_type="llm")
async def generate(query: str, context: str) → str:
    resp = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": f"{context}\n\n{query}"}]
    )
    return resp.choices[0].message.content
# Every call is now visible in LangSmith dashboard
```



LangSmith is the fastest path to LLM observability. Use it first. Migrate to OpenTelemetry + Grafana when you need more customization or self-hosting.



AI Coach John

Follow



PROIBRIDGE
Strive For Better Future

 Repost

Observability Stack Comparison

Tool	Best For	Cost	Setup
LangSmith	LLM-native: prompts, evals, runs	Free tier + paid	3 env vars
OpenTelemetry	Standardized traces, any backend	Open source	Moderate
Datadog LLM	Enterprise, existing Datadog users	High	Agent-based
Arize Phoenix	LLM evals, RAGAS, embeddings viz	Open source	Low
Grafana + Tempo	Self-hosted, full control	Infra only	High
Prometheus	Metrics, latency, cost counters	Open source	Low

👍 Recommended Stack for Most Teams

Start: LangSmith (zero-config, LLM-native, free tier). Add: Prometheus + Grafana for custom dashboards and alerts. Evaluate: Arize Phoenix for RAGAS quality metrics visualization.



AI Coach John

Follow



↺ Repost

LLM Observability **Mistakes**

- ✖ **Tracing only the LLM call, not retrieval.** Retrieval is often the bottleneck. A slow FAISS search looks fine if you only measure generation time.
- ✖ **No cost attribution by feature.** Without feature tagging on every LLM call, you cannot explain a cost spike to your finance team or make informed optimization decisions.
- ✖ **No prompt version in traces.** When quality changes, you have no way to correlate the change with a specific prompt version without this tag in every trace.
- ✖ **Relying only on user thumbs-up/down.** Most users never leave feedback. You need automated RAGAS scores as a signal that runs without user participation.
- ✖ **Mixing production and dev traces in the same project.** Dev traces pollute your production quality metrics. Always use separate LangSmith projects per environment.
- ✖ **No golden query regression set.** Without 50 known-good queries, you cannot detect when a deploy silently degrades existing behavior.



LLM Observability **Full Reference**

Pillar	Implementation	Alert Threshold
Distributed traces	OpenTelemetry spans per component	Missing span in any trace
Cost per feature	LLMCallRecord.compute_cost() + tag	Daily budget exceeded
Latency per component	timed() context manager	p99 above 2x 7-day baseline
Prompt versioning	SHA-256 hash logged in trace	Unrecognized hash in prod
Quality evaluation	RAGAS weekly on 100 queries	Faithfulness below 0.80
User satisfaction	Feedback linked to trace_id	Negative rate above 15%
Quick start	LangSmith (3 env vars, zero code)	Set up before first deploy

 **Trace every request. Attribute every token. Version every prompt. You cannot fix what you cannot see.**



AI Coach John

Follow



 Repost

 KEY TAKEAWAY

Observability Is Not Optional in Production

Every LLM failure that takes hours to debug is an observability gap. Every cost overrun that surprises your finance team is a missing attribution tag. Every quality regression that users find before you do is a missing metric.

✓ Start Today: 3 Steps

- Set up LangSmith with 3 env vars and the @traceable decorator on your LLM calls
- Add feature tagging to every LLM call and generate a daily cost report
- Run RAGAS faithfulness evaluation weekly on 100 sampled production queries

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost